

🔒 ТЕХНИЧЕСКИЙ ПАСПОРТ И ИБ-АНАЛИЗ КОНТУРА КОНФИГУРАЦИИ (config.js)

Настоящий технический документ содержит детальное описание, профили безопасности и пошаговый структурный разбор системных перехватчиков сетевых HTTPS-транзакций.

--- Схема рантайм-валидации сетевых ответов и очистки сессий ---

ВЫЗОВ СЕТЕВОГО МЕТОДА: `secureFetch(url, options)`

↓ Инъекция: `options.headers['Authorization'] = `Bearer ${token}``

ИСПОЛНЕНИЕ ЗАПРОСА И ПЕРЕХВАТ ОТВЕТА СЕРВЕРА

Условие перехватчика затвора: `if (response.status === 401)`

↓ Сверка состояния ОЗУ: `const hasToken = localStorage.getItem('pg_access_token')`

КОНТУР А: Сессия протухла в процессе работы

✗ Атомарное выжигание невалидных ключей из памяти браузера.
🗑️ `removeItem('pg_access_token');`
🔄 `window.location.reload();` → Сброс в чистый экран входа.

КОНТУР Б: Холодный старт без токена

😴 Мягкое бесшумное переключение контейнеров вкладок.
👁️ `loginContainer.classList.remove('d-none');`
👤 `appContainer.classList.add('d-none');`

ФИНАЛ: RETURN RESPONSE → ПЕРЕДАЧА ОТВЕТА ВЫЗЫВАЮЩЕМУ МОДУЛЮ

🛡️ Профиль ИБ-защиты подсистемы `secureFetch`:

- **Защита от фиксации невалидных сессий (Session Hijacking):** Перехватчик статуса `401` надежно защищает систему от попыток операторов выполнять действия с протухшими, отозванными или подмененными JWT-токенами. При обнаружении `401 Unauthorized` Контур А немедленно выжигает из памяти `pg_access_token` и `panel_user_role`. Команда `reload()` полностью сбрасывает ОЗУ вкладки браузера, гарантируя, что старый токен невозможно извлечь и использовать повторно.
- **Предотвращение бесконечных циклов циклического редиректа:** Контур Б разделяет ситуации, когда токен отсутствовал изначально (например, при первом открытии вкладки в браузере). Вместо вызова деструктивной перезагрузки страницы, метод мягко переключает классы видимости Bootstrap (`d-none`), выводя форму входа без создания избыточной нагрузки на веб-сервер Nginx.

Централизованный фильтр вывода уведомлений, обеспечивающий принудительную санитизацию входящих текстовых сообщений и блокирующий проведение атак класса Cross-Site Scripting (XSS).

--- Схема очистки строковых примитивов от HTML-инъектов ---

ВХОДНОЙ СТРОКОВЫЙ ПРИМИТИВ: showAlert(message)



ЭТАП А: ПРИНУДИТЕЛЬНАЯ РЕГУЛЯРНАЯ ЗАЧИСТКА ТЕГОВ (REGEXP)

```
message.replace(/<[^>]*>/g, ' ').replace(/\s+/g, ' ').trim();
```



ЭТАП Б: ВЫЗОВ СТРОГОГО СИСТЕМНОГО ДИАЛОГОВОГО ОКНА БРАУЗЕРА

alert(cleanMessage);

Выполнение скриптов внутри нативного модального окна физически изолировано от DOM-дерева

ФИНАЛ: ВЫВОД И СТИРАНИЕ ВРЕМЕННОГО СТРОКОВОГО БУФЕРА

Профиль ИБ-защиты подсистемы санитизации:

- **Тотальное вырезание вредоносных HTML-тегов (XSS Clean):** На строке 46 внедрен жесткий регулярный ИБ-фильтр `message.replace(/<[^>]*>/g, ' ')`. Любые попытки передать скрытый деструктивный JS-код со стороны бэкенда или перехваченных ответов СУБД на корню аннигилируются. Конвертер превращает теги в безопасные пробельные символы, вырезая вредоносную нагрузку из рантайма страницы.
- **Изоляция контекста выполнения (Песочница Alert):** Вывод очищенных уведомлений принудительно замкнут на нативную системную функцию браузера `alert(cleanMessage)`. Нативные окна оперируют чистыми строковыми примитивами и не выполняют HTML-парсинг `innerHTML`, что полностью изолирует их от DOM-дерева. Даже если тег будет деформирован, браузер отобразит его как плоский текст, исключая уязвимости Stored/Reflected XSS.
- **Нормализация пробельных символов (DoS-String Protection):** Каскадное сжатие множественных пробелов и символов табуляции `replace(/\s+/g, ' ')` страхует буфер рендеринга модального окна от попыток сдвинуть границы интерфейса гигантскими массивами пустых строк.